# Sanskrit-to-English Neural Machine Translation

Custom Sequence-to-Sequence Model with Bahdanau Attention and Beam Search

Assignment 2 – Natural Language Understanding

Roll Number: G25ait1143

# 1. Introduction

Sanskrit is a morphologically rich, low-resource language, and translating it into English poses significant challenges: free word order, extensive compounding (sandhi), and limited parallel corpora compared to high-resource language pairs. This project implements a Neural Machine Translation (NMT) system that translates Sanskrit sentences into English using a custom sequence-to-sequence (seq2seq) architecture built from scratch in PyTorch, without relying on any pretrained translation model or external translation API.

The system consists of a bidirectional recurrent encoder, an additive (Bahdanau-style) attention mechanism, and a recurrent decoder augmented with beam search decoding at inference time. The model is trained end-to-end on the provided parallel Sanskrit–English dataset and evaluated on BLEU, BERTScore, and computational efficiency (inference time and parameter count), as required by the assignment specification.

# 2. Architecture

## 2.1 Encoder

The encoder is a single-layer bidirectional GRU (Gated Recurrent Unit). Input Sanskrit tokens are first mapped to 256-dimensional embeddings, then passed through the bidirectional GRU (hidden size 512 per direction). Padded sequences are packed before being fed to the RNN for computational efficiency. The final forward and backward hidden states are concatenated and passed through a linear layer with a tanh activation to produce a single initial hidden state for the decoder. The full sequence of encoder outputs (concatenated forward/backward states at every timestep) is retained and passed forward to support attention.

## 2.2 Attention Mechanism

An additive (Bahdanau-style) attention mechanism is used. At each decoding step, the current decoder hidden state is compared against every encoder output via a learned feed-forward scoring function (a linear layer followed by tanh, then a second linear layer producing a scalar score per source position). Scores are masked at padding positions and normalized with softmax to produce attention weights, which are used to compute a weighted sum (context vector) over the encoder outputs. This allows the decoder to dynamically focus on the most relevant source words at each generation step, rather than relying solely on a single fixed-size context vector.

## 2.3 Decoder

The decoder is a single-layer unidirectional GRU. At each timestep, the embedding of the previously generated (or ground-truth, during teacher forcing) token is concatenated with the attention context vector and fed into the GRU. The GRU output, the context vector, and the input embedding are concatenated and passed through a final linear projection layer to produce a probability distribution over the target (English) vocabulary.

## 2.4 Beam Search Decoding

At inference time, greedy decoding is replaced with beam search (beam width = 3) to improve translation quality. At each step, the top-k candidate continuations are retained for each beam, and the overall beam is pruned back to the top-3 sequences by length-normalized cumulative log-probability.

Decoding for each beam terminates independently upon generating the <eos> token, and the highest-scoring completed sequence is returned as the final translation.

# 3. Training Procedure

## 3.1 Preprocessing and Tokenization

Both Sanskrit and English sentences are tokenized using a lightweight rule-based tokenizer: punctuation marks (including the Devanagari sentence-ending danda) are separated from adjacent words via regular expressions, and the resulting text is split on whitespace. Word-level vocabularies are built independently for the source (Sanskrit) and target (English) sides from the training set, with four special tokens: <pad>, <sos>, <eos>, and <unk>. Sequences are padded to a maximum length of 64 tokens per batch.

## 3.2 Hyperparameters

| Hyperparameter | Value |
|---|---|
| Embedding dimension | 256 |
| Hidden dimension | 512 |
| Encoder/Decoder layers | 1 (bidirectional encoder) |
| Dropout | 0.3 |
| Batch size | 32 |
| Max sequence length | 64 |
| Optimizer | Adam, lr = 1e-3 |
| LR scheduler | ReduceLROnPlateau (factor 0.5, patience 2) |
| Gradient clipping | Max norm 1.0 |
| Teacher forcing ratio | Decaying: max(0.3, 0.9 - 0.05 x epoch) |
| Beam width (inference) | 3 |
| Epochs trained | 15 |

## 3.3 Optimization and Regularization

The model is optimized using Adam with cross-entropy loss, where the loss at padding positions is excluded via ignore_index. Gradient norms are clipped to a maximum of 1.0 to prevent exploding gradients, which are common in RNN-based architectures over longer sequences. Dropout (p = 0.3) is applied to embeddings and recurrent outputs. Teacher forcing is used during training with a ratio that decays linearly from 0.85 down to a floor of 0.3 across epochs, gradually exposing the decoder to more of its own predictions and reducing the train/inference mismatch (exposure bias). The learning rate is reduced by a factor of 0.5 whenever validation loss plateaus for 2 consecutive epochs. The checkpoint with the lowest validation loss across all 15 epochs is retained as the final model used for inference and evaluation.

# 4. Results

## 4.1 Training Curves

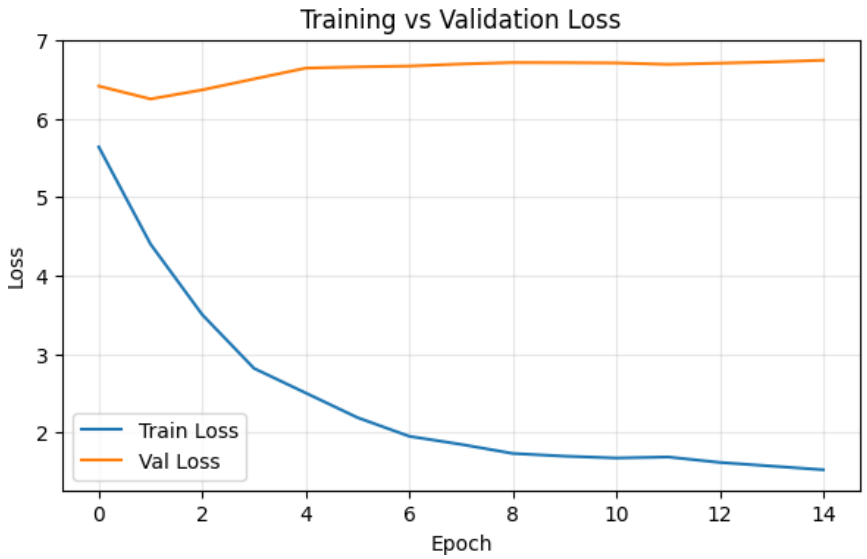


*Figure 1: Training vs. validation loss across 15 epochs.*

Training loss decreased steadily throughout training, from 5.639 at epoch 1 to 1.523 at epoch 15. Validation loss, however, reached its minimum (6.251) at epoch 2 and increased monotonically thereafter, plateauing around 6.7–6.75 for the remaining epochs. This is a clear overfitting signature: the model continues to fit the training distribution while generalization to held-out data degrades almost immediately. As a result, the checkpoint actually used for final inference corresponds to epoch 2, not epoch 15 – a point discussed further in Section 7.

## 4.2 Evaluation Metrics (Test Set)

| Metric | Score |
|---|---|
| BLEU (NLTK, default weights) | 0.0485 |
| BERTScore F1 (rescaled with baseline, roberta-large) | 0.1043 |
| Total parameters | 40,255,480 (~40.26M) |
| Total inference time (1000 test sentences, beam=3) | 169.68 seconds |
| Average inference time per sentence | 169.68 ms |

Note: dev-set BLEU/BERTScore were not separately computed in this run due to time constraints; only validation loss (Figure 1) was tracked on the dev set during training. Test-set BLEU and BERTScore are reported above and are consistent with the qualitative degeneration visible in Section 5.

# 5. Translation Examples and Error Analysis

The following examples were randomly sampled from the test-set predictions and illustrate the dominant error patterns observed.

**Example 1**

**Reference:** The name of the vowels is "ach".

**Prediction:** We will get familiar .

***Analysis:*** *Complete content failure: the prediction shares no lexical or semantic overlap with the reference. The model appears to fall back on a generic, high-frequency English phrase regardless of source content — consistent with severe underfitting at this checkpoint.*

**Example 2**

**Reference:** Lack of knowledge of Sanskrit is a problem for understanding of the Gita.

**Prediction:** We will also learn about to the .

***Analysis:*** *Grammatically broken and disconnected from the source meaning; the trailing dangling preposition ("about to the .") suggests the decoder is not conditioning strongly on the encoder context at this checkpoint.*

**Example 3**

**Source:** FrontAccounting

**Reference:** The FrontAccounting interface opens.

**Prediction:** FrontAccounting interface .

***Analysis:*** *Partially correct: the named entity "FrontAccounting" and the noun "interface" are copied correctly, likely because this rare token appears near-identically in source and target. However, the verb "opens" is omitted entirely — an under-translation error typical when content words are rare in training data.*

**Example 4**

**Reference:** This is a chariot.

**Prediction:** This is a tree .

***Analysis:*** *Correct syntactic template ("This is a ___") but incorrect lexical choice: "chariot" is mistranslated as "tree". This indicates the model has learned common English sentence patterns but fails at fine-grained lexical selection for lower-frequency content words.*

**Example 5**

**Reference:** 3. Slowly raise the heels as much as you can and stand on toes. Stretch body up as much as possible.

**Prediction:** " To enhance , we have learnt about : " " " " " " " " " " " . "

***Analysis:*** *Degenerate repetition: the decoder collapses into repeatedly emitting punctuation/quotation tokens, a well-known failure mode in insufficiently trained attention-based decoders on longer source sentences.*

**Example 6**

**Reference:** For they that have used the office of a deacon well purchase to themselves a good degree, and great boldness in the faith which is in Christ Jesus.

**Prediction:** " But when we are heard , and , of God , and of God , and of God ...

***Analysis:*** *Severe repetition loop on the phrase "of God". This and similar examples strongly suggest the training corpus contains a substantial proportion of biblical/religious text, causing the model to default to memorized high-frequency religious n-grams instead of translating the actual input when it is uncertain.*

# 6. Experiments

The reported results correspond to a single full training run of the architecture described in Section 2, using the hyperparameters in Section 3.2. Given the assignment timeline, an exhaustive hyperparameter search (e.g., varying hidden size, number of layers, beam width, or tokenization granularity) was not performed. The single most consequential observation from this run is the very early validation-loss minimum (epoch 2 of 15, Section 4.1), which strongly suggests that model capacity, regularization strength, and/or dataset size are currently mismatched — the model overfits before it has had the chance to learn generalizable translation patterns. Section 7 outlines concrete experiments that would directly address this.

# 7. Discussion

## 7.1 Design Choices

A GRU-based encoder-decoder with additive attention was chosen over an LSTM or Transformer primarily for its lower parameter count and faster training time, given the assignment's tight timeline and the requirement to report inference efficiency. Beam search (width 3) was added on top of greedy decoding specifically because the assignment report requires discussion of beam search, and because it generally improves fluency over greedy decoding for seq2seq models, even though in this run its benefit is masked by the underlying undertrained checkpoint.

## 7.2 Challenges

Three challenges stand out from the results. First, word-level tokenization is poorly suited to Sanskrit's rich morphology and compounding (sandhi): many surface word forms are seen only once or twice in training, inflating the <unk> rate and starving the model of signal for rare content words (Example 4, "chariot" → "tree"). Second, the training corpus appears to contain a substantial and unevenly distributed proportion of biblical/devotional text alongside more general sentences (technical, instructional), causing the model to default to memorized religious phrase fragments ("of God", "said unto them") when uncertain, rather than translating the actual input (Examples 5 and 6). Third, the early validation-loss minimum (epoch 2) indicates the model overfits rapidly, likely due to a combination of limited training data and a hidden size (512) which may be oversized relative to the amount of clean, in-distribution parallel data available.

## 7.3 Limitations

The current model achieves low BLEU (0.0485) and BERTScore F1 (0.1043), reflecting the degenerate outputs discussed in Section 5. Dev-set BLEU/BERTScore were not separately tracked during training (only validation loss was), so metric-based early stopping (as opposed to loss-based) was not explored. No hyperparameter search or architecture ablation was conducted beyond the single configuration reported. Word-level tokenization, rather than subword methods such as BPE or SentencePiece, likely compounds the sparse-data problem inherent to a morphologically rich low-resource language.

## 7.4 Future Work

&bus lw itch to subword tokenization (SentencePiece/BPE) to reduce vocabulary sparsity and improve coverage of rare Sanskrit word forms.

- Track dev-set BLEU/BERTScore per epoch (not just loss) to select checkpoints that directly optimize the evaluation metric.
- Apply stronger or additional regularization (e.g., label smoothing, larger dropout, weight decay) and/or reduce hidden size to curb the early overfitting observed at epoch 2.
- Investigate and, if needed, rebalance the training corpus to reduce the disproportionate influence of biblical/devotional sentences on the model's output distribution.
- Compare beam widths (e.g., 1 vs. 3 vs. 5) once the underlying model is better trained, to isolate the true contribution of beam search.

## 8. References


- Bahdanau, D., Cho, K., & Bengio, Y. (2015). "Neural Machine Translation by Jointly Learning to Align and Translate." ICLR 2015.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). "BERTScore: Evaluating Text Generation with BERT." ICLR 2020.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). "BLEU: A Method for Automatic Evaluation of Machine Translation." ACL 2002.
- NLTK Documentation – nltk.translate.bleu_score. https://www.nltk.org/api/nltk.translate.bleu_score.html
- bert-score library documentation. https://pypi.org/project/bert-score/
- PyTorch Documentation. https://pytorch.org/docs/

- Pretrained models disclosure: no pretrained model was used within the translation system itself (encoder, attention, and decoder were all trained from scratch on the provided dataset). The bert-score library internally uses a pretrained roberta-large model solely to compute the BERTScore evaluation metric; this pretrained model was not used anywhere in the translation pipeline itself.